

Aprendizaje de Máquina

Sesión 3: Flujo de un proyecto de Machine Learning

Angel Rodríguez

Avisos

- ▶ ¡Ya llegó el libro!
- ▶ Anuncio y la liga en blackboard.

<https://research-ebsco-com.udlap.idm.oclc.org/plink/0bc248dc-1ced-32f8-a529-0a165bb8feaa>

Objetivos de la sesión

- ▶ Comprender las etapas principales de un proyecto de aprendizaje de máquina.
- ▶ Identificar la diferencia entre entrenamiento, validación y prueba.
- ▶ Reconocer la importancia de la preparación de datos antes de entrenar modelos.
- ▶ Introducir el concepto de *pipeline* de Machine Learning.
- ▶ Implementar un flujo básico usando Scikit-Learn.

Agenda de la sesión

1. Repaso: tipos de aprendizaje y tipos de problema.
2. ¿Qué significa construir un proyecto de Machine Learning?
3. Preparación de datos.
4. División de datos: entrenamiento, validación y prueba.
5. Entrenamiento y evaluación inicial.
6. Introducción a pipelines en Scikit-Learn.

Idea central

Un modelo no empieza en el algoritmo: empieza en el **problema**, los **datos** y la forma en que vamos a **evaluar** los resultados.

Repaso: ¿qué tipo de problema tengo?

Antes de elegir un algoritmo, debemos clasificar el problema:

Tipo de problema	Pregunta principal
Clasificación	¿A qué categoría pertenece esta observación?
Regresión	¿Qué valor numérico se espera?
Agrupamiento	¿Qué observaciones se parecen entre sí?
Reducción de dimensión	¿Cómo representar datos complejos con menos variables?

Ejemplo: no es lo mismo predecir si un cliente comprará un producto que predecir cuánto gastará.

De la pregunta al modelo

Un proyecto de aprendizaje de máquina puede verse como una cadena de decisiones:



Punto importante

Si el problema está mal definido o los datos no son adecuados, el algoritmo difícilmente resolverá el proyecto.

Flujo general de un proyecto de Machine Learning

Un flujo típico incluye las siguientes etapas:

1. Definir el problema y la métrica de éxito.
2. Obtener y explorar los datos.
3. Preparar los datos.
4. Dividir los datos en conjuntos de entrenamiento, validación y prueba.
5. Seleccionar y entrenar modelos.
6. Evaluar resultados.
7. Ajustar hiperparámetros.
8. Reportar conclusiones y limitaciones.

Nota: en la práctica, este flujo no siempre es lineal; muchas veces se regresa a etapas anteriores.

Etapa 1: definir el problema

Antes de programar, debemos responder:

- ▶ ¿Cuál es la variable que queremos predecir o analizar?
- ▶ ¿Es un problema supervisado o no supervisado?
- ▶ ¿Es clasificación, regresión o agrupamiento?
- ▶ ¿Qué significa que el modelo sea “bueno”?
- ▶ ¿Qué errores serían más costosos?

Ejemplo

No basta decir: “queremos analizar clientes”.

Una formulación más clara sería: “queremos predecir si un cliente cancelará su servicio durante el próximo mes”.

Etapa 2: obtener y explorar los datos

La exploración inicial busca entender qué contienen los datos.

Pregunta	Ejemplo
¿Cuántas observaciones hay?	número de filas
¿Qué variables existen?	numéricas, categóricas, texto
¿Hay datos faltantes?	valores nulos o vacíos
¿Hay valores atípicos?	edades negativas, precios extremos
¿Hay desbalance?	95 % clase A, 5 % clase B

Objetivo: detectar problemas antes de entrenar modelos.

Exploración rápida en Python

Algunas instrucciones útiles para comenzar:

```
import pandas as pd

df = pd.read_csv("datos.csv")

print(df.shape) # filas y columnas
print(df.head()) # primeras observaciones
print(df.info()) # tipos de datos
print(df.describe()) # resumen numerico
print(df.isna().sum()) # datos faltantes
```

Pregunta para discutir

¿Qué problemas podrían aparecer si entrenamos un modelo sin revisar primero los datos?

Etapa 3: preparación de datos

La preparación de datos transforma una tabla cruda en una representación útil para el modelo.

- ▶ Limpieza de valores faltantes.
- ▶ Codificación de variables categóricas.
- ▶ Escalamiento de variables numéricas.
- ▶ Separación entre variables predictoras X y variable objetivo y .
- ▶ Construcción de nuevas variables cuando sea necesario.

Advertencia

La preparación debe aprenderse usando solamente los datos de entrenamiento. De lo contrario, podemos introducir **fuga de información**.

Datos faltantes

Los datos faltantes pueden tratarse de varias formas:

Estrategia	Comentario
Eliminar filas	simple, pero puede perder información
Eliminar columnas	útil si hay demasiados faltantes
Imputar media/mediana	común en variables numéricas
Imputar moda	común en variables categóricas
Agregar indicador	útil si “estar faltante” aporta información

Ejemplo: el ingreso faltante en una solicitud de crédito puede no ser aleatorio.

Variables categóricas

Muchos algoritmos necesitan datos numéricos. Por eso, las categorías deben codificarse.

Variable original	Posible codificación
color = rojo, azul, verde	one-hot encoding
ciudad = Puebla, CDMX, Monterrey	variables binarias por ciudad
nivel = bajo, medio, alto	codificación ordinal, si existe orden real

Cuidado

No todas las categorías tienen un orden. Codificarlas como 1, 2, 3 puede introducir relaciones falsas.

Escalamiento de variables

Algunos modelos son sensibles a la escala de las variables.

Variable	Rango típico
Edad	18 a 90
Ingreso anual	50,000 a 2,000,000
Número de compras	0 a 100

Si no escalamos, una variable con valores grandes puede dominar métodos basados en distancias o gradientes.

Métodos comunes: estandarización y normalización.

¿Por qué dividir los datos?

Queremos estimar cómo funcionará el modelo con datos nuevos.

Conjunto	Uso principal
Entrenamiento	ajustar los parámetros del modelo
Validación	comparar modelos y ajustar hiperparámetros
Prueba	estimar el desempeño final con datos no usados antes

Regla práctica

El conjunto de prueba debe tocarse lo menos posible. No debe usarse para tomar decisiones durante el desarrollo.

División de datos



Una división común puede ser 60 %-20 %-20 % o 70 %-15 %-15 %, dependiendo del tamaño del conjunto de datos.

Pregunta

¿Qué pasaría si evaluamos el modelo con los mismos datos con los que fue entrenado?

División de datos en Scikit-Learn

Un ejemplo simple:

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)
```

- ▶ `X_train, y_train`: entrenamiento.
- ▶ `X_val, y_val`: validación.
- ▶ `X_test, y_test`: prueba final.

Entrenamiento y predicción

Entrenar un modelo significa ajustar sus parámetros usando datos de entrenamiento.

Entrenar: datos → modelo ajustado

Después, el modelo se usa para hacer predicciones sobre datos nuevos.

Prededir: nuevas observaciones → respuestas
estimadas

Entrenamiento básico en Scikit-Learn

Ejemplo con regresión logística:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_val)
acc = accuracy_score(y_val, y_pred)

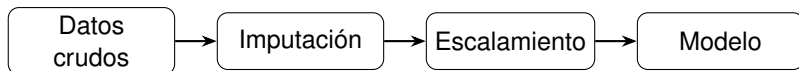
print("Accuracy:", acc)
```

Interpretación

Primero entrenamos con `fit`; después evaluamos predicciones con `predict`.

¿Qué es un pipeline?

Un *pipeline* es una secuencia ordenada de pasos para transformar datos y entrenar un modelo.



Ventajas

Reduce errores, hace el código más reproducible y evita aplicar transformaciones inconsistentes.

Pipeline básico en Scikit-Learn

Ejemplo con escalamiento y regresión logística:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

pipe.fit(X_train, y_train)
score = pipe.score(X_val, y_val)

print("Validation score:", score)
```

Idea: el pipeline encapsula preparación y modelo en un solo objeto.

Actividad en clase

Trabajaremos con un conjunto de datos pequeño para construir un primer flujo completo.

Instrucciones:

1. Cargar el conjunto de datos.
2. Separar X y y .
3. Dividir en entrenamiento, validación y prueba.
4. Crear un pipeline con escalamiento y modelo.
5. Entrenar el modelo.
6. Evaluar en validación.
7. Registrar conclusiones iniciales.

Producto esperado

Un notebook ejecutable con el flujo completo y una breve interpretación de resultados.

Actividad

Vamos a suponer que nos contrataron para un proyecto en una compañía de bienes raíces. Vamos a realizar 8 pasos:

1. Entender el problema
2. Conseguir los datos
3. Explorar y visualizar los datos
4. Preparar los datos
5. Seleccionar un modelo y entrenarlo
6. Afinar el modelo
7. Presentar la solución
8. Lanzar, monitorear y mantener el sistema

1. Plantear el problema y ver el panorama general

- ▶ Define el objetivo en términos de negocio.
- ▶ Aclara cómo se usará la solución.
- ▶ Identifica soluciones actuales o atajos existentes.
- ▶ Decide cómo encuadrar el problema: supervisado/no supervisado, online/offline, etc.
- ▶ Define cómo se medirá el desempeño.
- ▶ Verifica que la métrica esté alineada con el objetivo de negocio.
- ▶ Establece el desempeño mínimo necesario para lograr el objetivo.
- ▶ Busca problemas comparables y herramientas reutilizables.
- ▶ Confirma si hay experiencia humana disponible.
- ▶ Describe cómo resolverías el problema manualmente.
- ▶ Lista los supuestos propios y de otras personas.
- ▶ Valida los supuestos siempre que sea posible.

2. Obtener los datos

Automatiza tanto como sea posible para poder obtener datos frescos fácilmente.

- ▶ Lista los datos necesarios y en qué cantidad.
- ▶ Encuentra y documenta de dónde obtenerlos.
- ▶ Estima cuánto espacio ocuparán.
- ▶ Revisa obligaciones legales y consigue autorización si hace falta.
- ▶ Obtén permisos de acceso.
- ▶ Crea un espacio de trabajo con almacenamiento suficiente.
- ▶ Descarga o recopila los datos.
- ▶ Convierte los datos a un formato manipulable sin alterar su contenido.
- ▶ Elimina o protege información sensible; por ejemplo, anonimízala.
- ▶ Revisa tamaño y tipo: series de tiempo, muestra, geográficos, etc.
- ▶ Separa un conjunto de prueba y nunca lo mires: evita el *data snooping*.

3. Explorar los datos para obtener insights

Intenta obtener insights de una persona experta en el dominio.

- ▶ Crea una copia para exploración; muestrea si es necesario.
- ▶ Usa un notebook para registrar la exploración.
- ▶ Estudia cada atributo:
 - ▶ nombre y tipo;
 - ▶ porcentaje de valores faltantes;
 - ▶ ruido, outliers o errores de redondeo;
 - ▶ utilidad potencial para la tarea;
 - ▶ distribución: gaussiana, uniforme, logarítmica, etc.
- ▶ En aprendizaje supervisado, identifica el/los atributos objetivo.
- ▶ Visualiza los datos.
- ▶ Estudia correlaciones entre atributos.
- ▶ Revisa cómo resolverías el problema manualmente.
- ▶ Identifica transformaciones prometedoras.
- ▶ Detecta datos extra que serían útiles; si hace falta, vuelve a obtener datos.
- ▶ Documenta lo aprendido.

4. Preparar los datos para exponer mejor los patrones

Trabaja sobre copias y conserva intacto el dataset original. Escribe funciones para todas las transformaciones: reutilizarlas, aplicarlas a datos nuevos y tratarlas como hiperparámetros será mucho más fácil.

Limpieza y selección

- ▶ Corrige o elimina outliers, si conviene.
- ▶ Rellena valores faltantes con cero, media, mediana, etc.; o elimina filas/columnas.
- ▶ Elimina atributos que no aportan información útil para la tarea.

Ingeniería de características

- ▶ Discretiza variables continuas cuando sea útil.
- ▶ Descompón variables: categorías, fecha/hora, texto, etc.

Transformaciones y escalado

- ▶ Agrega transformaciones prometedoras: $\log(x)$, $\sqrt{x}$, x^2 , etc.
- ▶ Combina o agrega atributos para crear nuevas variables.
- ▶ Estandariza o normaliza las características.

Diseño reproducible

- ▶ Prepara fácilmente nuevos datasets frescos.
- ▶ Limpia de forma consistente el set de prueba y nuevas instancias.
- ▶ Reutiliza transformaciones en proyectos futuros.

5. Explorar muchos modelos y preseleccionar los mejores

Si los datos son enormes, entrena con muestras más pequeñas para probar muchos modelos, sabiendo que esto puede penalizar modelos complejos. Automatiza todo lo posible.

- ▶ Entrena modelos rápidos de varias familias: lineales, naive Bayes, SVM, Random Forests, redes neuronales, etc.
- ▶ Usa parámetros estándar al inicio.
- ▶ Mide y compara desempeño.
- ▶ Para cada modelo, usa validación cruzada N-fold y calcula media y desviación estándar.
- ▶ Analiza variables importantes para cada algoritmo.
- ▶ Estudia los tipos de errores que cometen los modelos.
- ▶ Pregunta qué datos habría usado una persona para evitar esos errores.
- ▶ Haz una ronda rápida de selección e ingeniería de características.
- ▶ Itera una o dos veces.
- ▶ Preselecciona los tres a cinco modelos más prometedores, idealmente con errores distintos.

6. Afinar modelos y combinarlos en una gran solución

Usa la mayor cantidad de datos posible en esta etapa, especialmente al final del afinamiento. Automatiza cuando puedas.

Afinamiento

- ▶ Ajusta hiperparámetros con validación cruzada.
- ▶ Trata las decisiones de transformación como hiperparámetros, sobre todo si no estás seguro: imputar con cero, mediana o eliminar filas.
- ▶ Si hay muchos valores, prefiere búsqueda aleatoria sobre grid search.
- ▶ Si entrenar tarda mucho, considera optimización bayesiana.

Ensamble y evaluación final

- ▶ Prueba métodos de ensamble: combinar buenos modelos suele superar a cada modelo individual.
- ▶ Cuando confíes en el modelo final, mide el desempeño en el conjunto de prueba.
- ▶ Estima el error de generalización.
- ▶ No ajustes el modelo después de medir el error de generalización: sobreajustarías el conjunto de prueba.

7. Presentar la solución

- ▶ Documenta lo que hiciste.
- ▶ Crea una presentación clara y atractiva.
- ▶ Empieza destacando el panorama general.
- ▶ Explica por qué la solución cumple el objetivo de negocio.
- ▶ Presenta hallazgos interesantes del proceso.
- ▶ Describe qué funcionó y qué no funcionó.
- ▶ Enumera supuestos y limitaciones del sistema.
- ▶ Comunica hallazgos clave con visualizaciones memorables o frases fáciles de recordar.

8. Lanzar, monitorear y mantener el sistema

Lanzamiento

- ▶ Prepara la solución para producción.
- ▶ Conéctala a las entradas de datos productivas.
- ▶ Escribe pruebas unitarias y verificaciones operativas.

Monitoreo

- ▶ Crea código para revisar desempeño en vivo a intervalos regulares.
- ▶ Dispara alertas cuando el desempeño caiga.

Mantenimiento

- ▶ Vigila degradación lenta: los modelos se deterioran conforme evolucionan los datos.
- ▶ Considera que medir desempeño puede requerir revisión humana o crowdsourcing.
- ▶ Monitorea también la calidad de las entradas: sensores fallando, datos obsoletos de otro equipo, etc.
- ▶ Reentrena modelos regularmente con datos frescos y automatiza tanto como sea posible.

0. Datasets disponibles

- ▶ Google Datasets Search
- ▶ Hugging Face Datasets
- ▶ OpenML.org
- ▶ Kaggle.com
- ▶ UC Irvine Machine Learning Repository
- ▶ Stanford Large Network Dataset Collection
- ▶ Amazon's AWS datasets
- ▶ U.S. Government's Open Data
- ▶ DataPortals.org
- ▶ Wikipedia's list of machine learning datasets

1. Entender el problema.

- ▶ Utilizaremos el dataset “California Housing Prices” del repositorio StatLib.
- ▶ Tarea: Usando los datos, crear un modelo para predecir los precios de vivienda en cualquier distrito.
- ▶ Tendremos varias propiedades: población, sueldo promedio y precio de vivienda promedio para un distrito (600 a 3000 personas).
- ▶ Métrica de desempeño: *raíz del error cuadrático medio* (RMSE por sus siglas en inglés):

$$\text{RMSE}(\mathbf{X}, \mathbf{y}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(\mathbf{x}^{(i)}) - y^{(i)})^2} \quad (1)$$

Notaciones

- ▶ m es el número de instancias.
- ▶ $\mathbf{x}^{(i)}$ es un vector con los valores de las características (sin la etiqueta) de la i -ésima instancia en el data set.
- ▶ $y^{(i)}$ es la etiqueta de la i -ésima instancia en el dataset.
- ▶ $\mathbf{X}$ es la matriz que contiene todas las características (sin las etiquetas) de **todas** las instancias del conjunto de datos (o dataset). Cada renglón contiene las características de la instancia, y el i -ésimo renglón equivale a la transpuesta de $\mathbf{x}^{(i)}$, denotado por $(\mathbf{x}^{(i)})^\top$.

$$\mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ \vdots \\ (\mathbf{x}^{(m)})^\top \end{pmatrix} \quad (2)$$

- ▶ h es nuestra función de predicción, llamada *hipótesis*. $\hat{y}^{(i)}$ es la predicción para la i -ésima observación: $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$.

Notaciones

- ▶ *itálicas* minúsculas para escalares (como m , $y^{(i)}$).
- ▶ **negritas** minúsculas para vectores (como $\mathbf{x}^{(i)}$).
- ▶ **negritas** mayúsculas para matrices (como $\mathbf{X}$).

Otra métrica de desempeño

Si hay muchos *outliers* (valores atípicos), es mejor utilizar otras métricas ya que RMSE es sensible a los outliers. Ejemplo:

Error absoluto medio (MAE por sus siglas en inglés):

$$\text{MAE}(\mathbf{X}, \mathbf{y}, h) = \frac{1}{m} \sum_{i=1}^m |h(\mathbf{x}^{(i)}) - y^{(i)}| \quad (3)$$

Para todo fin práctico, estamos midiendo la distancia entre dos vectores:

- ▶ Distancia euclideana (o ℓ_2): RMSE
- ▶ Distancia Manhattan (o ℓ_1): MAE
- ▶ Se puede usar en general cualquier $\ell_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$.

2. Conseguir los datos

► `https://hom1.info/colab-p`

3. Explorar y visualizar los datos

Viendo los histogramas:

- ▶ La ganancia media no está en USD. La normalizaron para que vaya de 0.5 a 15.
- ▶ Valor y edad de alojamiento están "truncados".
Principalmente el valor es el más importante, **¿porqué?**
 - ▶ Se puede pedir que se vuelva a recolectar la información.
 - ▶ Eliminar esos distritos del conjunto de entrenamiento y de prueba.
- ▶ Todos los atributos tienen diferentes escalas.
- ▶ Los histogramas están sesgados a la derecha.

4. Preparar los datos

Continúa la próxima clase...

5. Seleccionar un modelo y entrenarlo

6. Afinar el modelo

7. Presentar la solución

8. Lanzar, monitorear y mantener el sistema

¡Gracias!